Author:Ganesh Babu G

dplyr & tidyverse

Complete Verb & Helper Functions Reference

ROW VERBS

filter() / filter_out() — Keep or drop rows matching a condition

Helper Function	Purpose
`between(x, l, r)`	Detects if values fall in a numeric range
`near(x, y)`	Compares floating-point values with tolerance
`if_any(.cols, .fns)`	Returns TRUE if any column matches the condition
`if_all(.cols, .fns)`	Returns TRUE if all columns match the condition
`na_if(x, y)`	Converts specific values to NA
`coalesce(...)`	Returns the first non-missing value
`case_when(...)`	Multi-condition vectorised if-else
`replace_when(...)`	Variant of case_when for replacements
`if_else(cond, t, f)`	Simple vectorised if-else

arrange() — Order rows using column values

Helper Function	Purpose
`desc(x)`	Sort column in descending order
`across()`	Arrange by multiple computed columns

distinct() — Keep unique/distinct rows

No dedicated helpers — commonly paired with `select()` to distinct on a subset of columns.

slice() family — Subset rows using their positions

Function	Purpose
`slice(n)`	Pick rows by explicit index
`slice_head(n=)`	Select first n rows
`slice_tail(n=)`	Select last n rows
`slice_min(col, n=)`	Rows with the smallest values
`slice_max(col, n=)`	Rows with the largest values
`slice_sample(n=)`	Random sample of n rows

Positional helpers used inside slice:

Helper Function	Purpose
`n()`	Total number of rows in the current group

`row_number()`	Rank rows by position
`order_by()`	Controls ordering in window functions

COLUMN VERBS

select() — Keep or drop columns by name or type

Tidy-select helper functions (most extensive set):

Helper Function	Purpose
`starts_with('x')`	Columns whose names start with a prefix
`ends_with('x')`	Columns whose names end with a suffix
`contains('x')`	Columns whose names contain a literal string
`matches('regex')`	Columns whose names match a regex pattern
`num_range('x', 1:3)`	Columns like x1, x2, x3
`all_of(vars)`	All columns in a character vector (strict — errors if missing)
`any_of(vars)`	Any columns in a character vector (lenient — no error)
`everything()`	All remaining columns not yet selected
`last_col(offset)`	The last column, with optional offset
`where(fn)`	Columns where a predicate is TRUE, e.g. where(is.numeric)
`c(...)`	Combine multiple tidy-select expressions
`group_cols()`	Select existing grouping variables

mutate() / transmute() — Create, modify, or delete columns

Helper Function	Purpose
`across(.cols, .fns)`	Apply function(s) to multiple columns at once
`c_across(cols)`	Combine column values row-wise (with rowwise())
`pick(cols)`	Select a column subset as a data frame
`if_else(cond, t, f)`	Vectorised conditional assignment
`case_when(...)`	Multi-condition if-else for complex logic
`replace_when(...)`	Variant of case_when for value replacement
`coalesce(...)`	First non-NA value across columns
`na_if(x, y)`	Replace a specific value with NA
`recode_values()`	Recode specific values to new values
`replace_values()`	Replace specific values
`lag(x, n)`	Lagged (previous) values
`lead(x, n)`	Leading (next) values
`cumall(x)`	Cumulative all()
`cumany(x)`	Cumulative any()
`cummean(x)`	Cumulative mean

`row_number()`	Rank by row position
`min_rank(x)`	Minimum rank (ties get lowest rank)
`dense_rank(x)`	Dense rank (no gaps between ranks)
`ntile(x, n)`	Bucket values into n equal groups
`percent_rank(x)`	Proportional ranking (0 to 1)
`cume_dist(x)`	Cumulative distribution rank
`consecutive_id(...)`	Unique ID for consecutive value combinations
`between(x, l, r)`	Detect if values fall within a range

rename() / rename_with() — Rename columns

Helper Function	Purpose
`rename_with(.fn, .cols)`	Apply a function (e.g. toupper, str_replace) to rename columns

relocate() — Change column order

Uses tidy-select helpers: `starts_with()`, `where()`, `everything()`. Also accepts `.before` and `.after` arguments.

pull() — Extract a single column as a vector

Supports negative integer position, e.g. `pull(df, -1)` for the last column.

GROUP VERBS

group_by() / ungroup() — Group data for subsequent operations

Helper / Argument	Purpose
`across()`	Group by computed values
`.add = TRUE`	Add new groups to existing grouping
`group_cols()`	Select existing grouping columns in tidy-select

summarise() / summarize() — Reduce each group to a single row

Helper Function	Purpose
`n()`	Count rows in the current group
`n_distinct(x)`	Count unique values
`first(x)`	First value in the group
`last(x)`	Last value in the group
`nth(x, n)`	The nth value in the group
`mean() / median()`	Base R central tendency aggregates
`sum() / min() / max()`	Base R aggregates
`sd() / var()`	Standard deviation / variance
`across(.cols, .fns)`	Summarise multiple columns simultaneously

`cur_group()`	Current group as a tibble
`cur_group_id()`	Unique integer ID of the current group
`cur_group_rows()`	Row indices of the current group
`cur_column()`	Current column name (inside across())

count() / tally() / add_count() / add_tally() — Count observations per group

Argument / Helper	Purpose
`wt = col`	Weighted count by a numeric column
`sort = TRUE`	Sort results by descending count
`name = 'col'`	Rename the output count column

rowwise() — Treat each row as its own group

Helper Function	Purpose
`c_across(cols)`	Combine values across columns within each row

reframe() — Transform each group to an arbitrary number of rows

Like `summarise()` but allows returning multiple rows per group. No dedicated helpers beyond those shared with summarise.

TWO-TABLE (JOIN) VERBS

Mutating Joins

Function	Purpose
`inner_join(x, y)`	Keep rows with matches in both tables
`left_join(x, y)`	Keep all rows from x, match from y
`right_join(x, y)`	Keep all rows from y, match from x
`full_join(x, y)`	Keep all rows from both tables

Filtering Joins

Function	Purpose
`semi_join(x, y)`	Keep rows in x that have a match in y
`anti_join(x, y)`	Keep rows in x that have NO match in y

Other Joins

Function	Purpose
`nest_join(x, y)`	Nest matching y rows into a list-column in x
`cross_join(x, y)`	Cartesian product of all row combinations

Author:Ganesh Babu G

Universal Join Helper

Helper Function	Purpose
`join_by(x == y)`	Equi-join specification
`join_by(x >= y)`	Inequality join
`join_by(closest(x >= y))`	Rolling join (nearest match)
`join_by(between(x, l, r))`	Overlap / range join

Set Operations

Function	Purpose
`intersect(x, y)`	Rows in both x and y
`union(x, y)`	All unique rows from x and y
`union_all(x, y)`	All rows from x and y (duplicates kept)
`setdiff(x, y)`	Rows in x but not y
`setequal(x, y)`	TRUE if x and y have same rows
`symdiff(x, y)`	Rows in x or y but not both

Bind Operations

Function	Purpose
`bind_rows(...)`	Stack data frames vertically
`bind_cols(...)`	Combine data frames side by side

MULTI-COLUMN HELPERS (pair with any verb)

Function	Used With	Purpose
`across(.cols, .fns)`	mutate, summarise, filter, group_by	Apply function(s) to multiple columns
`if_any(.cols, .fns)`	filter	TRUE if any column satisfies condition
`if_all(.cols, .fns)`	filter	TRUE if all columns satisfy condition
`c_across(cols)`	mutate + rowwise	Combine columns within each row
`pick(cols)`	mutate, summarise	Pass a column subset as a data frame

GROUPING HELPER FUNCTIONS (Experimental)

Function	Purpose
`group_map(.f)`	Apply function to each group — returns a list
`group_modify(.f)`	Apply function to each group — returns a tibble
`group_walk(.f)`	Apply function to each group for side effects
`group_trim()`	Remove unused factor levels from grouping structure
`group_cols()`	Select grouping variables inside tidy-select context

Author:Ganesh Babu G

TIDY-SELECT HELPERS (Reusable Across Verbs)

These work inside `select()`, `rename_with()`, `relocate()`, `across()`, `pick()`, and more.

Helper	Description
`starts_with('x')`	Match column names beginning with prefix
`ends_with('x')`	Match column names ending with suffix
`contains('x')`	Match column names containing literal string
`matches('regex')`	Match column names by regular expression
`num_range('x', 1:3)`	Match numbered columns: x1, x2, x3
`all_of(vars)`	All columns from character vector (strict)
`any_of(vars)`	Any columns from character vector (lenient)
`everything()`	All remaining unselected columns
`last_col(offset)`	Last column with optional backward offset
`where(fn)`	Columns satisfying a predicate (e.g. where(is.numeric))
`group_cols()`	Currently active grouping columns

ROWS_* VERBS (Manipulate Individual Rows)

Function	Purpose
`rows_insert(x, y)`	Insert new rows (error if key exists)
`rows_append(x, y)`	Append rows regardless of key conflicts
`rows_update(x, y)`	Update matching rows with new values
`rows_patch(x, y)`	Update only NA values in matching rows
`rows_upsert(x, y)`	Insert or update rows based on key match
`rows_delete(x, y)`	Delete rows matching keys in y